PROGRAMMING IN HASKELL

Topic 13.2– XML to Markdown with xml-conduit

XML → Markdown - Our workflow

What We're Building

Input

A books.xml catalogue with nested authors, ISBNs & publish dates.

Haskell Pipeline

xml-conduit Cursors parse XML into typed records. Pure functions render each Book to Markdown.

Output

One catalogue.md ready for rendering to pdfs etc.

The XML Catalogue Structure

XML Listing

```
<catalogue>
    <book isbn="0-00-000">
        <title>...</title>
        <subtitle>...</subtitle>
        <authors>
            <author id="a1">...</author>
        <published year="2024"/>
    </book>
</catalogue>
```

Key Parsing Notes

isbn

Attribute on <book> — not a child element. Use attribute.

subtitle

Optional child — parse with childText + fmap.

authors

Nested container: element "author" inside <authors>.

year

Attribute on <published> — drill two levels then read attr.

Haskell Data Model

Haskell

```haskell
data Author = Author
  { authorId   :: String
  , authorName :: String
  } deriving (Show)
```

Haskell

```haskell
data Book = Book
  { isbn      :: String
  , title     :: String
  , subtitle  :: Maybe String  -- optional
  , authors   :: [Author]
  , pubYear   :: String
  } deriving (Show)
```

Author

Flat record

id is parsed from an XML attribute, name from element text content.

[Author] list

Natural fit for zero-or-more <author> children inside <authors>.

Book

Maybe String for subtitle

Captures the optional <subtitle> element cleanly — Nothing if absent.

deriving (Show)

Automatic Show instances for quick debugging in GHCi.

xml-conduit: Core Concepts

Document & Element

```
import Text.XML
import Data.Map (Map)

-- Document contains the root Element
-- Element has:
--   Name, [Node],
--   Map Name Text (attributes)
```

Text.XML is the high-level API. Elements carry attributes as a Map.

Cursor API

```
import Text.XML.Cursor

-- fromDocument :: Document -> Cursor
-- element / content / attribute
-- ($/) ($//)
--   composable axis operators
```

Cursors let you navigate the tree with composable axis functions.

Parsing Entry Points

```
readFile def "books.xml"
  :: IO Document

parseLBS def lazyBS
  :: Either SomException Document

parseText def lazyText
  :: Either SomeException Document
```

def = default ParseSettings. Use readFile in IO, parseLBS for in-memory.

Axis and cursor -> Axis

- Fundamental part of how the XML tree is traversed.

```
type Axis = XmlTree -> [XmlTree]

.. Examples

getChildren :: Axis -- returns direct children of a node
getParents :: Axis -- returns the parent
getDescendants :: Axis -- returns all nodes below this one
```

- Because an axis returns a list, you can compose them with >>= or use them with `mapM`.

```
getChildren >>> hasName "book" >>> getAttrValue "isbn"
```

The >>> here is arrow composition — apply the first axis, then the second to each result, and so on

Axis and cursor -> Cursor

- Rather than a function, a cursor is a data structure that remembers where you are in the tree and how you got there — it stores both the current node and the surrounding context (parents, siblings) needed to navigate back out.

```
data Cursor = Cursor
      { current :: Node ,
        lefts :: [Node] -- siblings to the left ,
        rights :: [Node] -- siblings to the right ,
        parents :: [...] -- the path back up }
```

- With a cursor you can move in any direction and then move back:

```
child 0 cursor        -- move to first child
parent cursor         -- move back up
nextSibling cursor    -- move right
```

Cursor

```
<book>
    <title>Haskell Programming</title>
    <author>Someone</author>
    <price>40</price>
</book>
```

if your cursor is sitting on <book>, then:

```
child 0 cursor -- moves to <title>
child 1 cursor -- moves to <author>
child 2 cursor -- moves to <price>
child 3 cursor -- returns Nothing (no fourth child)
```

Difference

An axis is a **query** — you ask "what nodes are reachable from here in this direction?”
and get a list back. It is stateless and functional.

A cursor is a **position** — it is a value that represents where you are right now in the document, and you move it around. It is more like a pointer or iterator, but immutable and safe because the full context is encoded in the data structure.

In practice, axes suit bulk queries
("give me all <price> nodes in this document")

while cursors suit targeted navigation
("step into the third child, read its attribute, step back out").

Libraries like xml-conduit expose both styles and you often combine them.

Parsing the Catalogue with xml-conduit

Haskell

```haskell
import Text.XML          (readFile, def, Document)
import Text.XML.Cursor   (fromDocument, Cursor, ($/)
                         , element, content, attribute)
import Data.Text         (Text)
import qualified Data.Text as T

-- Helper: text of first matching child element
childText :: Name -> Cursor -> Maybe Text
childText n cur =
  listToMaybe $ cur $/ element n &/ content

-- Parse one <author> cursor
parseAuthor :: Cursor -> Author
parseAuthor cur = Author
  { authorId   = T.unpack . head . attribute "id" $ cur
  , authorName = T.unpack . T.concat $ cur $/ content }

-- Parse one <book> cursor
parseBook :: Cursor -> Book
parseBook cur = Book
  { isbn     = T.unpack . head $ attribute "isbn" cur
  , title    = maybe "" T.unpack (childText "title" cur)
  , subtitle = fmap T.unpack   (childText "subtitle" cur)
  , authors  = map parseAuthor
               (cur $/ element "authors" $/ element "author")
  , pubYear  = T.unpack . head $
               cur $/ element "published" >=> attribute "year" }

-- Entry point: Document -> [Book]
parseCatalogue :: Document -> [Book]
parseCatalogue doc = map parseBook $
  fromDocument doc $/ element "catalogue" &/ element "book"
```

Key Operators

`($/) axis op`

Descend into direct children matching the predicate. Composes naturally.

`element "name"`

Selects child elements by local name — the core traversal primitive.

`attribute "n"`

Reads a named attribute from the current element cursor.

`(>=>)`

composition — chains steps: find child, then read attr.

Rendering Books to Markdown

Haskell

```haskell
-- Render a single Author inline
renderAuthor :: Author -> String
renderAuthor a =
  "- **[" ++ authorId a ++ "]** "
  ++ authorName a

-- Render one Book as a Markdown section
renderBook :: Book -> String
renderBook b = unlines $
  [ "## " ++ title b
  , "| Field | Value |"
  , "|-------|-------|"
  , "| ISBN  | `" ++ isbn b ++ "` |"
  , "| Year  | " ++ pubYear b ++ " |"
  ] ++
  maybe []
    (\s -> ["| Sub | " ++ s ++ " |"])
    (subtitle b) ++
  [ "", "### Authors" ] ++
  map renderAuthor (authors b)
```

Haskell

```haskell
-- Build catalogue.md
renderCatalogue :: [Book] -> String
renderCatalogue books = unlines $
  [ "# Book Catalogue"
  , ""
  , show (length books)
      ++ " book(s)."
  , ""
  ] ++ map renderBook books

-- Write to a file
writeCatalogue :: FilePath -> [Book] -> IO ()
writeCatalogue path books =
  writeFile path (renderCatalogue books)
```

Sample Output

```
## Haskell Programming
| ISBN | `978-1593272838` |
| Year | 2010 |
### Authors
- **[a1]** Bryan O'Sullivan
```

maybe function- applies the supplied default value ([] in this case) if Nothing if returned

Wiring It All Together

Haskell

```
module Main where

import Text.XML          (readFile, def)
import Text.XML.Cursor   (fromDocument)
import Prelude hiding    (readFile)
import System.Environment (getArgs)

main :: IO ()
main = do
      doc   <- Text.XML.readFile def “books.xml”
      let books = parseCatalogue doc
      putStrLn $ "Parsed " ++ show (length books) ++ " book(s)."
      writeCatalogue books
      putStrLn $ "Written to " ++ mdPath
```

1. **Read XML**
   readFile parses the file into a Document, throwing on malformed XML.
2. **Parse**
   parseCatalogue walks the cursor tree → [Book].
3. **Render**
   renderCatalogue converts [Book] → Markdown String.
4. **Write**
   writeFile saves the result to the output path.

Extension: One .md File per Book

Haskell

```haskell
import System.Directory (createDirectoryIfMissing)
import System.FilePath  ((</>), (<.>))

-- Sanitise a title for use as a filename
safeFileName :: String -> String
safeFileName = map
  (\c -> if c `elem` " /\\:*?<>|" then '-' else c)

-- Write each book to its own Markdown file
writeBookFiles :: FilePath -> [Book] -> IO ()
writeBookFiles dir books = do
  createDirectoryIfMissing True dir
  mapM_ (writeOne dir) books

writeOne :: FilePath -> Book -> IO ()
writeOne dir b = do
  let name    = safeFileName (title b) <.> "md"
      outPath = dir </> name
  writeFile outPath (renderBook b)
  putStrLn $ "  Wrote: " ++ outPath
```

Key Functions

`createDirectoryIfMissing`

Creates the output directory (and parents) if it does not exist yet.

`safeFileName`

Strips characters invalid in file paths across platforms.

`mapM_`

Idiomatic way to perform an IO action for each item in a list.

```
$ ls output/
Haskell-Programming.md   Real-World-Haskell.md   Learn-You-a-Haskell.md
```

Summary

1. Define domain types (Book, Author) before writing any parser.
2. xml-conduit Cursors give composable, type-safe XML traversal.
3. Separate parsing from rendering — pure functions, easy to test.
4. Use Maybe for optional fields; handle missing data at the type level.
5. mapM_ / mapM make it trivial to process every book in the catalogue.
6. writeFile / createDirectoryIfMissing handle all file-system output.